02
Oggetti e classi

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2020/2021

Outline

Goal della lezione

- Illustrare i concetti base del paradigma object-oriented
- Mostrare un primo semplice programma Java
- Fornire una panoramica di alcuni meccanismi Java

Argomenti

- Oggetti e riferimenti
- Tipi primitivi
- Classi, metodi e campi
- Accenno a package e librerie
- Stampe a video
- Primo semplice programma Java

Outline

1. Elementi base dei tipi di Java
2. Principali costrutti dell'object-orientation
3. Programmi Java

"Everything is an object"

Riferimenti ad oggetti

- Nessun meccanismo per accedere ai dati per valore o puntatore!
- Le variabili conterranno dei riferimenti agli oggetti veri e propri, sono quindi dei nomi "locali" utilizzabili per denotare l'oggetto

```
// Creo un oggetto String, e lo assegno al nome s
// questo oggetto stringa rappresenta la sequenza
// vuota di caratteri
String s = new String();

// In generale si crea un oggetto con l'operatore <new>
// Accetta 0,1 o più argomenti, a seconda del tipo:
// valori primitivi (numeri), string literals, o oggetti
String s2 = new String("prova");
Point2D p = new Point2D(10.5,20.3);
Object o = new Object();
```

Variabili, oggetti, e valori primitivi

Concetti base

- **Variabile:** un contenitore con nome (come in C), usabile per denotare un oggetto
- **Valore primitivo:** p.e. un numero, anche assegnabile ad una variabile

```
String s = new String("prova");

// Un caso ad-hoc di Java, equivalente a:
// String s2 = new String("casa");
String s2 = "casa";

// Definisco il nome s4, ma non lo inizializzo
String s4;
// ora assegno s4, da qui in poi è utilizzabile
s4 = "auto";

// Il nome i è assegnato al valore primitivo 5
int i = 5;
```

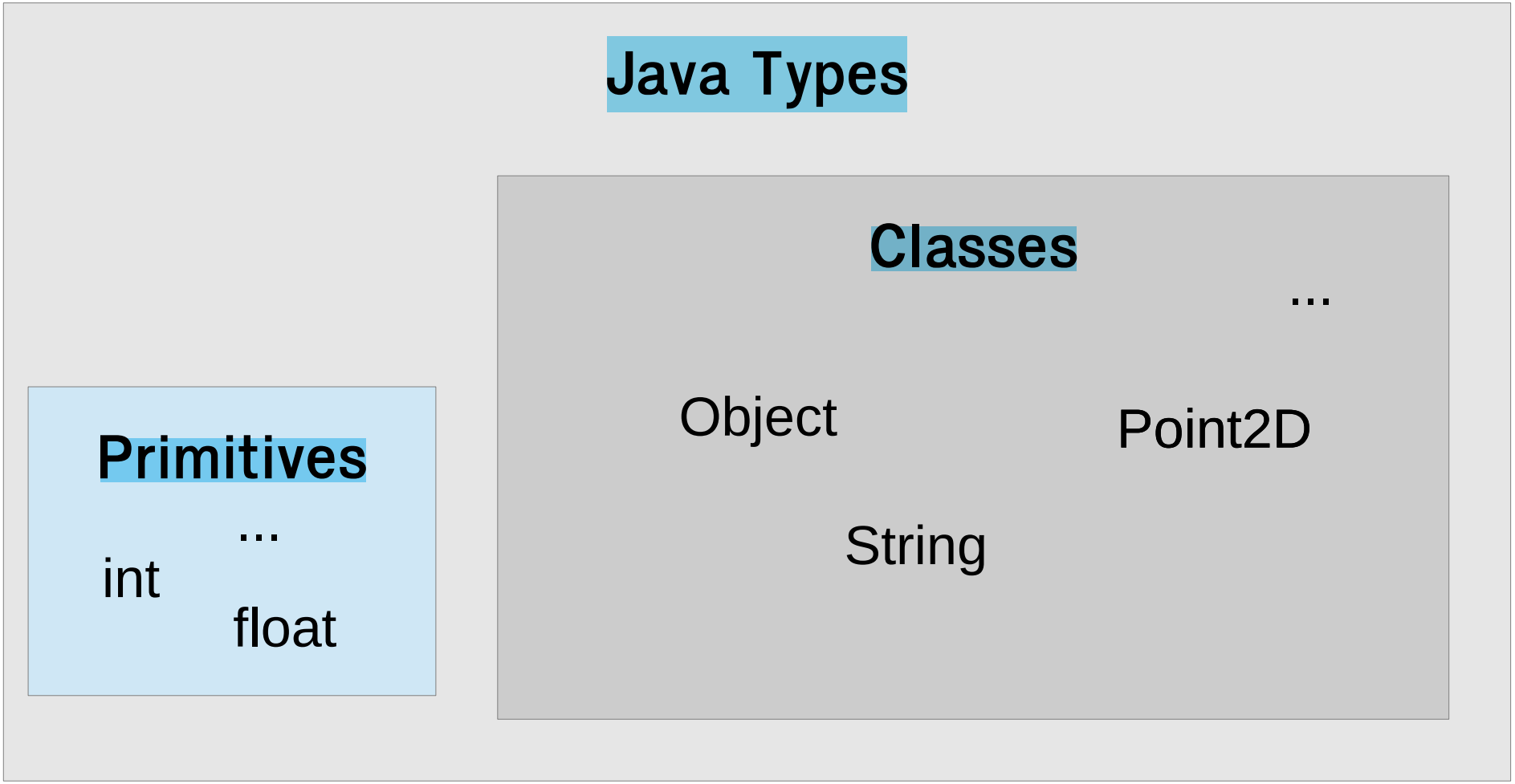
"(Almost) Everything is an object"

Tipi primitivi: tipi speciali usati per valori atomici

- Assomigliano molto a quelli del C, ma hanno dimensioni fissate
- I `boolean` possono valere `true` o `false`
- Altre classi di libreria (`BigDecimal`, `BigInteger`) gestiscono numeri di dimensione/precisione arbitraria

Primitive type	Size	Minimum	Maximum	Wrapper type
boolean	—	—	—	**Boolean**
char	16 bits	Unicode 0	Unicode 2^{16}- 1	**Character**
byte	8 bits	-128	+127	**Byte**
short	16 bits	-2^{15}	$+2^{15}-1$	**Short**
int	32 bits	-2^{31}	$+2^{31}-1$	**Integer**
long	64 bits	-2^{63}	$+2^{63}-1$	**Long**
float	32 bits	IEEE754	IEEE754	**Float**
double	64 bits	IEEE754	IEEE754	**Double**
void	—	—	—	**Void**

Una prima classificazione dei tipi



Variabili e tipi

```
int i = 5;                    // assegnamenti con tipi primitivi
double d = 145e-20;
boolean b = true;
int other = i;

Object o = new Object();      // assegnamenti con classi
String s = "altra stringa";
Point2D p = new Point2D(10.4,20.3);
Point2D q = p;                // assegnamento da/a variabile

Object on = null;             // assegnamento a null
```

Costrutto var: "local variable type inference" (da Java 10)

Costrutto var

- usabile nelle variabili locali (a funzioni/metodi) per avere maggiore concisione
- il compilatore capisce (inferisce) il tipo della varibile dall'espressione assegnata
- non abusarne, e comunque noi non lo useremo molto all'inizio del corso

```
var i = 5;                    // assegnamenti con tipi primitivi
var d = 145e-20;
var b = true;
var other = i;

var o = new Object();    // assegnamenti con classi
var s = "altra stringa";
var p = new Point2D(10.4,20.3);
var q = p;                // assegnamento da/a variabile
```

Oggetti e memoria

Gestione della memoria – entriamo nella JVM solo temporanemante

- tutti gli oggetti sono allocati nella memoria heap
- le variabili sono allocate nello stack, nei rispettivi record di attivazione
- le variabili di tipi primitivi contengono direttamente il valore
- le variabili che contengono oggetti in realtà hanno un riferimento verso lo heap
- nota: ancora non sappiamo cosa contiene un oggetto

Situazione relativa al codice della slide precedente

- le variabili `i`, `d`, `b`, `other` contengono valori
- tutte le altre contengono un riferimento ad un oggetto, nello heap
- solo `p` e `q` "(si) riferiscono (al)lo stesso oggetto"
- `on` contiene un riferimento speciale, `null`

Visibilità, ossia "scope", delle variabili

Definita una variabile, dove sarà visibile?

- Regole molto simili a quelle del C
- variabili dentro un blocco non sono visibili fuori
- differenza rispetto al C: variabili non inizializzate non sono utilizzabili!

Tempo di vita degli oggetti

- finita l'esecuzione delle istruzioni relative allo scope di una variabile, l'oggetto continua a esistere
- verrà deallocato automaticamente dal sistema se non più usato
  - se, direttamente o indirettamente, nessuna variabile lo può raggiungere
  - un componente della JVM, il garbage collector, è preposto a questo compito
  - ne vedremo il funzionamento a breve..

Outline

1. Elementi base dei tipi di Java
2. Principali costrutti dell'object-orientation
3. Programmi Java

Costruire classi

Premesse

- la classe è l'unità fondamentale di programmazione OO
- progettare e costruire classi correttamente sarà l'obbiettivo del corso
- incominciamo descrivendo la loro struttura generale
- daremo via via linee guida sempre più precise

Cos'è una classe

- è un template (stampino) per generare oggetti di una certa forma
- ne definisce tipo, struttura in memoria e comportamento

Classe vs. oggetto

- classe: è una descrizione (parte di programma)
- oggetto: è una entità a tempo di esecuzione, è istanza di una classe

Struttura di una classe

Nome della classe

è anche il nome del tipo

Proprietà della classe

Campi (o membri, o fields)

- descrivono la struttura/stato dell'oggetto in memoria

Metodi della classe

- descrivono i messaggi accettati e il comportamento corrispondente

(altri elementi che vedremo..)

Classi: un po' di codice

Costruzione classi

```
class A {    // A è il nome della classe
   ...       // qui si riporta il suo contenuto
}

class AltroEsempioDiClasse { // Nota il CamelCase
   ...
}
```

Uso

```
// codice cliente
A obj1 = new A();    // creo un oggetto di A, con nome obj1
A obj2 = new A();    // creo un altro oggetto di A
AltroEsempioDiClasse obj3 = new AltroEsempioDiClasse();
A obj4;              // variabile obj4 non inizializzata
obj4 = new A();      // ok
obj4 = new AltroEsempioDiClasse();// NO!! Errore semantico..
```

Campi

Elementi costitutivi dei campi

- i campi di una classe assomigliano ai membri di una struct del C
- ognuno è una sorta di variabile (nome + tipo)
  (per i campi non è usabile il costrutto `var`)
- ve ne possono essere 0,1, molti
- lo stato di un oggetto è l'attuale valore associato ai campi
- potrebbero essere valori primitivi, o altri oggetti

Valore di un campo

- impostabile al momento della sua dichiarazione
- se non inizializzato vale:
  - 0 per i tipi numerici
  - `false` per i booleani
  - `null` per le classi
- accessibile da codice cliente con notazione obj.field

Campi: un esempio "toy" (giocattolo)

Classe

```
class A {
  int i;
  int j = 2;
  Object o;
}
```

Uso

```
A obj = new A();
int a = obj.i;    // a varrà 0
int b = obj.j;    // b varrà 2
obj.i = 10;       // modifico lo stato di obj
int d = obj.i;    // d varrà 10
obj.o = new Object();  // riassegno obj.o
A obj2 = new A();
obj.i = obj2.i;  // quanto varrà ora obj.i? 0
```

Campi: un esempio realistico, Point3D

Classe

```
class Point3D {
  double x; // Nota, l'ordine dei campi è irrilevante
  double y;
  double z;
}
```

Uso

```
Point3D a = new Point3D();  // Creo due punti, di nome a e b
Point3D b = new Point3D();
a.x = 10.0; // Imposto lo stato di a
a.y = 20.0;
a.z = 30.0;
b.x = a.x * 2;  // Imposto lo stato di b
b.y = a.y * 2;  // .. a partire da quello di a
b.z = a.z * 2;
int mod2a = a.x * a.x + a.y * a.y + a.z * a.z;
int mod2b = b.x * b.x + b.y * b.y + b.z * b.z;
boolean aGreater = (mod2a > mod2b); // false
```

Metodi

Elementi costitutivi dei metodi

- i metodi di una classe assomigliano a funzioni (del C)
- ognuno ha una intestazione (signature) e un corpo (body)
  - a sua volta l'intestazione ha il nome, tipo di ritorno, argomenti
- di metodi ve ne possono essere 0,1, molti
- definiscono il comportamento dell'oggetto

Significato di un metodo

- codice cliente richiama un metodo con notazione obj.meth(args)
- ciò corrisponde a inviargli un messaggio
- `obj` è chiamato il receiver del messaggio (o della invocazione)
- il comportamento conseguente è dato dall'esecuzione del corpo
- il corpo può leggere/scrivere il valore dei campi

Metodi: esempio toy

Classe

```
class A {
  int i;
  void aggiungi(int a){ // input "int a"
    i = i + a;
  }
  int restituisciValore(){     // intestazione funzione
    return i;              // corpo funzione
  }
}
```

Uso

```
A obj = new A();
int v = obj.i;                  // vale 0
obj.aggiungi(10);               // modifico obj
obj.aggiungi(20);               // modifico obj
int v2 = obj.i;                 // vale 30
int v3 = obj.restituisciValore();  // vale 30
```

La variabile speciale this

this

- dentro ad un metodo si può accedere agli argomenti o ai campi
- per rendere meno ambigua la sintassi, Java fornisce una variabile speciale denotata con this, che contiene il riferimento all'oggetto che sta gestendo il messaggio corrente
- per motivi di leggibilità, è opportuno usarla sempre in questo corso

```
class A {
  int i;
  void aggiungi(int a){
    this.i = this.i + a;     // this.i: il "mio" campo i
  }
  int restituisciValore(){
    return this.i;
  }
  int get(){     // Un alias per restituisciValore
    return this.restituisciValore();
  }
}
```

Metodi: altro esempio Point3D

```
class Point3D {   // dichiarazione classe
  double x;       // 3 campi
  double y;
  double z;
  void build(double a,double b,double c){ // build
    this.x = a; this.y = b; this.z = c;
  }
  double getModuloQuadro(){                    // getModuloQuadro
    return this.x * this.x + this.y * this.y + this.z * this.z;
  }
  boolean equal(Point3D q){   // equal
    return this.x == q.x && this.y == q.y && this.z == q.z;
  }
}
..
// codice cliente
Point3D p1 = new Point3D();       // creo un punto p1
p1.build(10.0,20.0,30.0);         // ne imposto i valori
Point3D p2 = new Point3D();       // creo un punto p2
p2.build(10.0,20.0,31.0);         // ne imposto i valori
double m2 = p1.getModuloQuadro();// ottengo il modulo quadr. di p1
boolean b = p1.equal(p2);         // chiedo a p1 se è uguale a p2
```

Outline

1. Elementi base dei tipi di Java
2. Principali costrutti dell'object-orientation
3. Programmi Java

Programmi Java

Elementi costitutivi dei programmi Java

- librerie di classi del Java Development Kit
- librerie esterne (nostre o di altri)
- un insieme di classi che costituiscono l'applicazione
- almeno una di tali classi ha un metodo speciale `main`
- un `main` è il punto d'accesso di un programma

Il main

Il `main` deve avere la seguente dichiarazione:

- `public static void main(String[] args){..}`

tre concetti che spiegheremo in dettaglio nel prosieguo:

- `public` indica il fatto che debba essere visibile "a tutti"
- `static` indica che non è un metodo dell'oggetto, ma della classe
- `String[]` indica il tipo "array di stringhe"

Package e librerie Java: organizzazione

Librerie di Java

- Documentazione auto-generata, consultabile offline, o online:
  `https://docs.oracle.com/en/java/javase/11/docs/api/`
  (google search "`javadoc 11`")
- contano più di 4000 classi, raggruppate in 200+ package (e 50+ moduli)

Package

- un package è un contenitore di classi con uno scopo comune, di alto livello
- tipicamente un package contiene qualche decina di classi
- i package sono organizzati ad albero, con notazione `nome1.nome2.nome3`
- Package principali: `java.lang`, `java.util`, `java.io`

Package, moduli e librerie Java: moduli

Moduli (Java 9+)

- un modulo definisce un frammento di codice "autonomo":
  - testabile, distribuibile, con chiara interfaccia e dipendenze da altri
- p.e., `java.base`, `java.desktop`, `java.sql`, `java.xml`
- librerie esterne compatibili con Java 9+ sono distribuite in uno o più moduli
- un modulo è costituito internamente da uno o più package
- p.e., `java.base` contiene i package principali che useremo

Impatto sulla programmazione "base"

- il concetto di modulo non impatta i sorgenti, ma solo il "project management": per il momento non ce ne occuperemo perché il JDK fornisce "di default" accesso a tutti i moduli che ci servono (`java.*`)
- il concetto di package invece impatta i sorgenti: il nome completo di una classe dipende dal package in cui si trova

Package, moduli e librerie Java: uso

Importare una classe di "libreria"

- per usare le classi di una libreria prima le si importa
- lo si fa con la clausola `import`, da usare all'inizio del sorgente
  - importo la singola classe: `import java.util.Date;`
  - importo l'intero package: `import java.util.*;`
  - importazione di default: `import java.lang.*;`
- senza importazioni, ogni classe andrebbe sempre qualificata indicandone anche il package completo:

  ```
  java.util.Date obj = new java.util.Date();
  ```
- importare evita quindi solo di dover indicare ogni volta il package

Classi (e funzionalità) "deprecate"

- dichiarate come "scadute", ossia preferibilmente "da non usare più" (legacy)
- noi le utilizzeremo per scopi didattici
- p.e. `java.util.Date`

Stampe su schermo

La procedura di stampa `System.out.println`

- `System` è una classe nel package `java.lang`
- `out` è un suo campo statico, rappresenta lo standard output
- `println` è un metodo che accetta una stringa e la stampa
- l'operatore `+` concatena stringhe a valori

```
class A {
  int i;
  void stampa(){
    System.out.println("Sono un oggetto della classe A");
    System.out.println("Il mio valore di i è: " + this.i);
  }
}
..
A obj = new A();
obj.i = 12;
obj.stampa();
```

"Hello world" Java Program

Hello.java

```
// This class necessarily goes into a file Hello.java
class Hello{
  public static void main(String[] args){
    System.out.println("Hello World!");
  }
}
```

Compilazione ed esecuzione

- con un editor di testo si scrive la classe in un file `Hello.java`
- si compila la classe col comando: `javac Hello.java`
- se non ci sono errori, viene generato il bytecode `Hello.class`
- si esegua il programma con: `java Hello`
- la JVM cerca la classe `Hello`, e ne esegue il `main`

Librerie, oggetti, e stampe

```
import java.util.Date;
import java.util.Random;
// java.lang.System non va importata

class PrintingObjects{
  public static void main(String[] args){
    Date d = new Date();
    System.out.println("Data corrente: " + d);
    System.out.println("Millisec. dal 1/1/1970: " + d.getTime());

    Random r = new Random();
    System.out.println("Numero random: " + r.nextInt());
    System.out.println("Altro numero random: " + r.nextInt());
    System.out.println("N. random fra (0-99): " + r.nextInt(100));
    String vjava=System.getProperty("java.version");
    String osname=System.getProperty("os.name");
    String usrdir=System.getProperty("user.dir");
    System.out.println("Versione Java: " + vjava);
    System.out.println("OS Name: " + osname);
    System.out.println("Usr dir: " + usrdir);

  }
}
```

Il caso delle classi Wrapper dei tipi primitivi

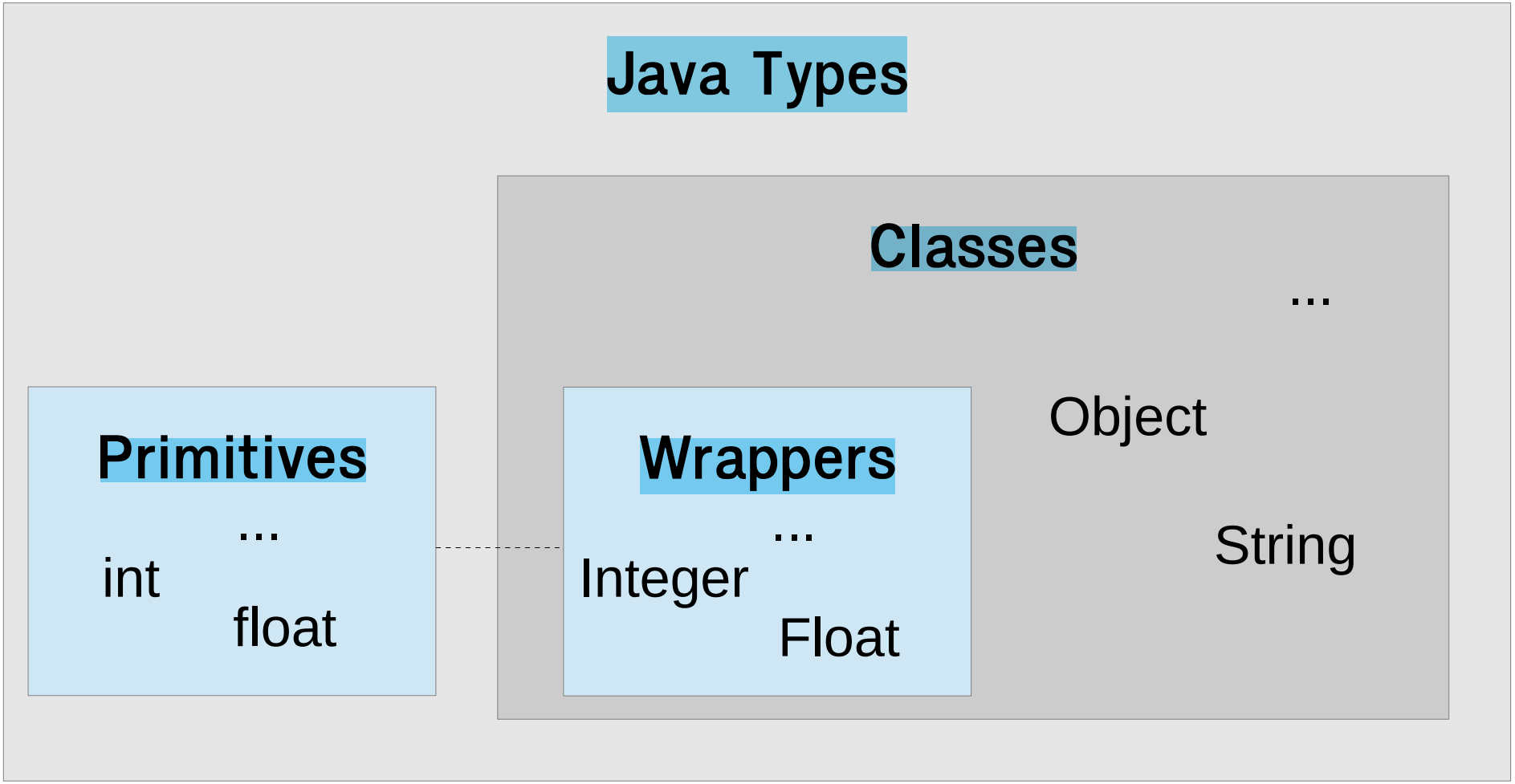
Le classi Wrapper

- ce ne è una per tipo primitivo, nel package `java.lang`
  - `Integer`, `Double`, `Boolean`,...
- hanno qualche metodo utile per fare conversioni
- poco usate per il momento

```
class Wrappers {
  public static void main(String[] s) {
    Integer i = new Integer(10);
    // nota gli escape character nella println
    System.out.println("Un intero \"wrapped\" " + i);
    // stampa: Un intero "wrapped" 10

    Double d = new Double(10.2);
    // nota la stampa formattata come nella printf del C.. poco usata in
     Java
    System.out.format("Un Double %f\n", d);
    // stampa: Un Double 10.200000
    System.out.println(d.intValue()); // 10
  }
}
```

Una nuova classificazione dei tipi



Costruire e provare classi

```
class Point3D {    // dichiarazione classe
  double x;        // 3 campi
  double y;
  double z;
  void build(double a,double b,double c){ // build
    this.x = a; this.y = b; this.z = c;
  }
  double getModuloQuadro(){                // getModuloQuadro
    return this.x * this.x + this.y * this.y + this.z * this.z;
  }
  boolean equal(Point3D q){    // equal
    return this.x == q.x && this.y == q.y && this.z == q.z;
  }
}
```

Classi, e classi clienti

```
class Point3D {   // dichiarazione classe
  double x;       // 3 campi
  double y;
  double z;
  void build(double a,double b,double c){ // build
    this.x = a; this.y = b; this.z = c;
  }
  double getModuloQuadro(){                // getModuloQuadro
    return this.x * this.x + this.y * this.y + this.z * this.z;
  }
  boolean equal(Point3D q){   // equal
    return this.x == q.x && this.y == q.y && this.z == q.z;
  }
}
```

```
class UsePoint3D{
  public static void main(String[] args){
    Point3D p1 = new Point3D();  // creo un punto p1
    p1.build(10.0,20.0,30.0);    // ne imposto i valori
    Point3D p2 = new Point3D();  // creo un punto p2
    p2.build(10.0,20.0,31.0);    // ne imposto i valori
    System.out.println("Modulo quadro di p1: " + p1.getModuloQuadro());
    System.out.println("p1 è uguale a p2? : " + p1.equal(p2));
  }
}
```

- si compilano separatamente, o con: `javac *.java`
- si esegue a partire dalla classe col `main`: `java UsePoint3D`

Preview del prossimo laboratorio

Obbiettivi

- familiarizzare con la compilazione da linea di comando in Java
- fare qualche esercizio con la costruzione e uso di classi